Project

Autonomous Software Agents

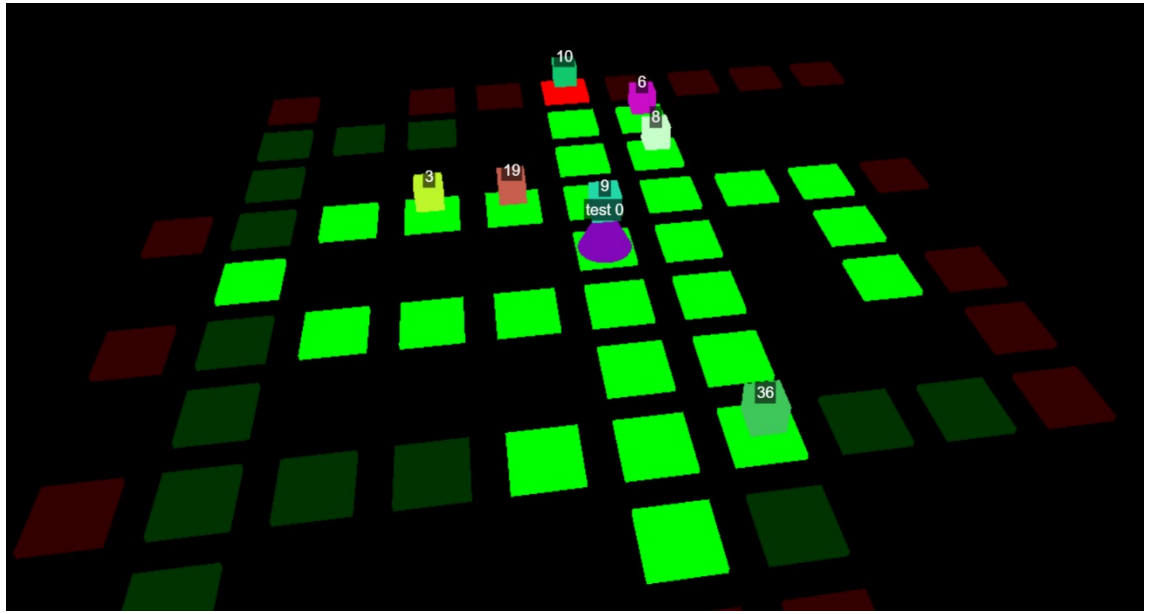
A.A. 2025-2026

Prof. Paolo Giorgini
Dr. Marco Robol
Dr. Marco Bombieri

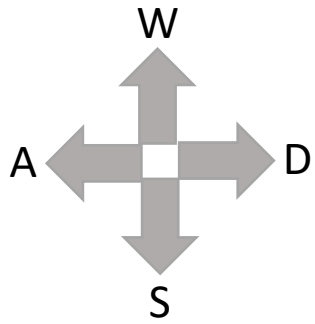


UNIVERSITY OF TRENTO - Italy
Department of Information
and Communication Technology

Deliveroo



Deliveroo.js is minimalistic parcels delivering web-based game developed specifically for this course and for educational purposes



Q: pick_up
E: put_down

© P. Giorgini and M. Robol and M. Bombieri

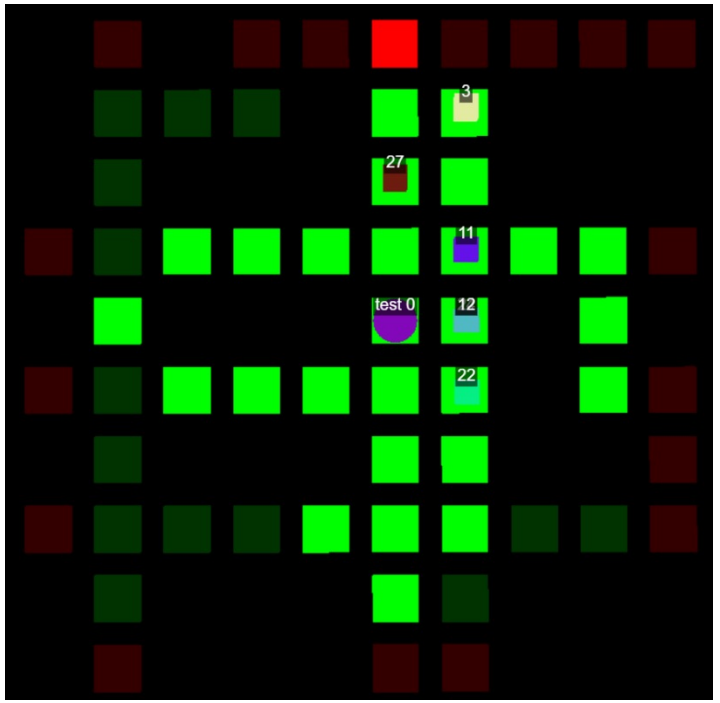
The objective of the project

- Develop an autonomous software that will play on your behalf
  - earning points by collecting as many parcels as possible and delivering them in the delivery zone
  - Using a BDI architecture or an LLM
    - Sensing the environment and managing Beliefs
    - Deliberating Intentions
    - Select plans from a library
    - Using an external planner component
    - Execute a plan (actions)
    - Defining strategies and behaviours
    - Replanning and redeliberating

© P. Giorgini and M. Robol and M. Bombieri

The Game – environment

- Grid MxN Tiles
  - 0: none - non walkable
  - 1: green - parcel-spawning
  - 2: red - delivery
  - 3: white - walkable
- Parcel
  - Located in a tile (x,y)
  - Appear in the grid with a Timer
  - More than one parcel at a time can appear during the game
  - Disappear when the timer expires or when it has been delivered
  - Can be picked up or put down in any free tile by a player
  - Can be carried by a player



Example map on a 10x10 grid as sensed by the ‘test’ player in the middle

© P. Giorgini and M. Robol and M. Bombieri

The Game - players

- **Players** can do:
  - Actions: {`move_right`, `move_left`, `move_down`, `move_up`, `pick_up`, `put_down`}
  - A player can pick up and carrying more than one parcel at a time
  - Sensing: acquiring information within a limited range (x_offset + y_offset < `sensing_distance`; shown in the game as light/shaded tiles)
    - `Parcel sensing: {id, x, y, carriedBy, reward}`
    - `Player sensing: {id, name, x, y, score}`
    - `Me: { id, name, x, y, score, penalty }`
- **The Game**
  - The objective for a player is to gain points picking up and delivering parcels
  - Each delivered parcel counts as much as its remaining time
  - Game Clock is used to update parcels' timer and duration of actions

© P. Giorgini and M. Robol and M. Bombieri

Actions

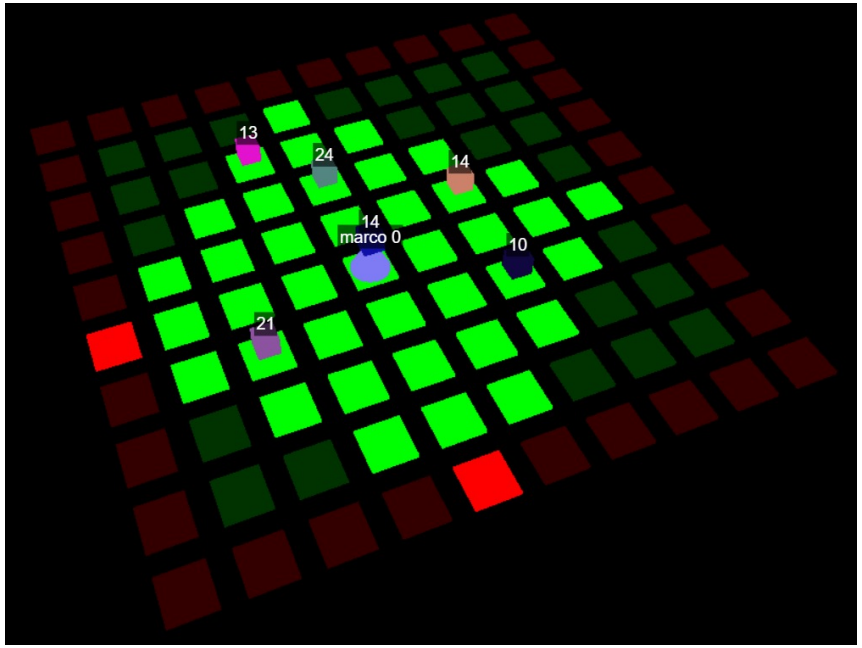
- Moving actions are not instantaneously
  - They have a fixed duration
    - When moving action starts, player is moved in between the initial and the target tile
      - By updating (increasing or decreasing) players' coordinate (X or Y) of 0.6
    - Then, when moving action ends, movement on the target tile is completed
      - By moving player of another 0.4
    - Therefore, it is possible to know whether a player is moving in one direction or another.
  - During the execution, starting and ending tiles are locked (they cannot be occupied by other players)
  - A player cannot move in a tile occupied by another player
    - The moving action will fail, and a penalty is assigned
  - Starting from (x,y) - right: x+1, left: x-1, up: y+1, down: y-1
- Pick up and put down are instantaneously
  - A player can pick up and put down a single parcel at a time
  - To pick up a parcel, the player has to be in the same tile of the parcel
  - A player can put down a parcel everywhere
    - The player gains points only putting down a parcel in a delivering area

© P. Giorgini and M. Robol and M. Bombieri

Sensing

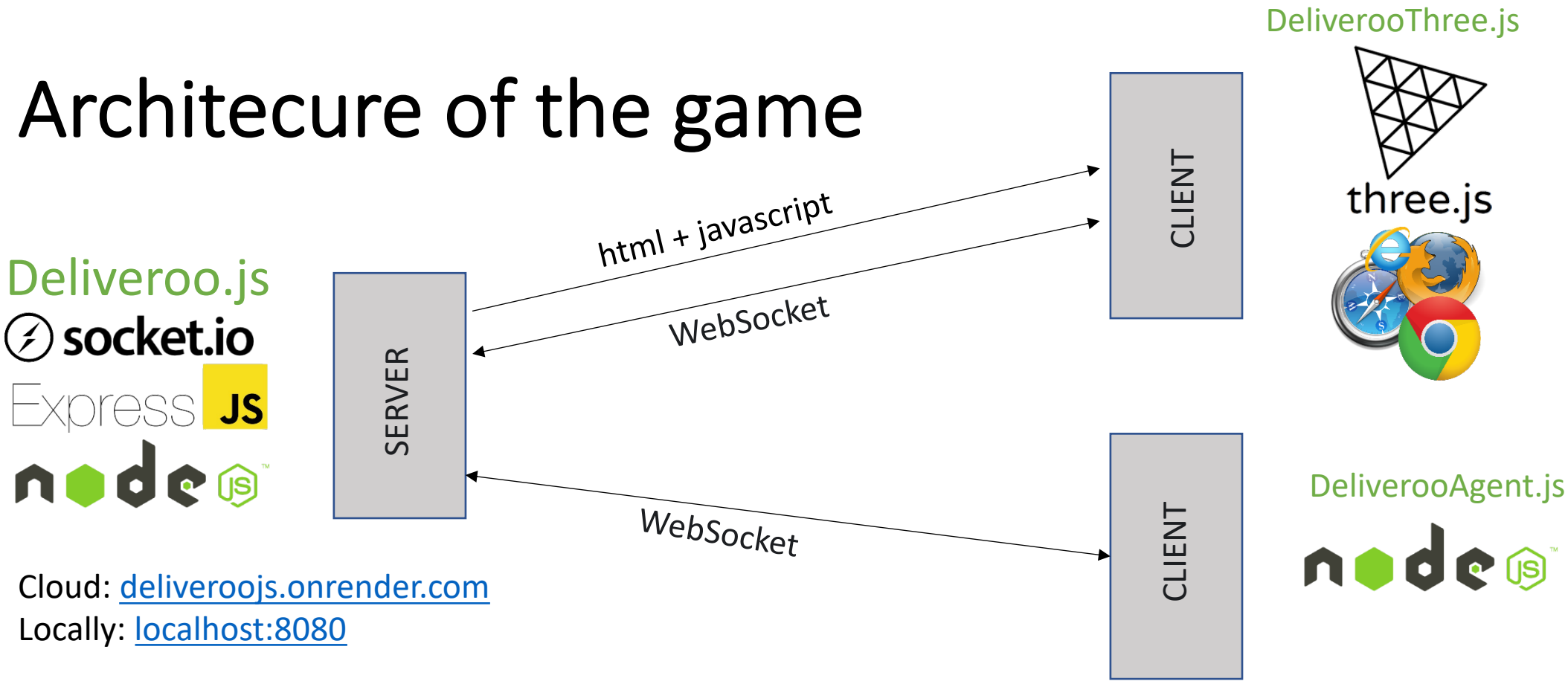
- On connection, the client receives game map as a list of tiles
  - `tile: {x, y, type}`
    - `Type '0': none - non walkable`
    - `Type '1': green - parcel-spawning`
    - `Type '2': red - delivery`
    - `Type '3': white - walkable`
- Then, every time something happens (parcel timer decades, player moves,…) the client receives sensing information for each tile that can be observed, on agents and parcels
  - `Parcel sensing {x, y, parcel:{id,carriedBy,reward}}`
  - `Agent sensing: {x, y, agent:{id,name,score} }`
  - `You: {id, name, x, y, score}`
- Everything outside from the sensed area cannot be known
  - Position of previously observed players can be guessed
  - Reward timer of parcels can be computed locally

Sensing is limited to area within:
x_offset + y_offset < 5



© P. Giorgini and M. Robol and M. Bombieri

Architecure of the game



Cloud: deliveroojs.onrender.com
Locally: localhost:8080

- Deliveroo.js is a web application based on Node.js and Socket.IO.
- Connection between client and server is based on Socket.IO and WebSocket.
- DeliverooThree.js is a 3D client that can be played directly from the browser.
  - Useful to test game strategies and see how the game is eveolving
  - No game logic is coded in the client, core of the game is on the server.

© P. Giorgini and M. Robol and M. Bombieri

Game authentication

- A token is needed to connect to the game
  - Every unique token is associated to one player
  - Tokens can be easily created through the 3D client, by specifying a name for the player. Just follow the instructions appearing in the browser when connecting to the game for the first time. Different players with different tokens can still have the same name (the id is different)
    - Tokens are signed by the game using a passphrase and may not be usable across different instances
    - Tokens do not expires
- Use the same token on your agent and 3D client:
  - Follow your agent (script) playing from its very perspective
- No limit on the number of tokens, get as many as you want
  - When accessing the game with the 3D client, the browser store your tokens in the local storage, to facilitate log-in procedure. You can still remove or replace them
  - In the case of no clients connected for a given token, after 10 seconds the NPC is removed from the grid until reconnection
  - Multiplayer mode is possible over the network (and even from the same computer)
    - Connect to the same game host and use different tokens
    - If the same token is used, players will control the very same character

© P. Giorgini and M. Robol and M. Bombieri

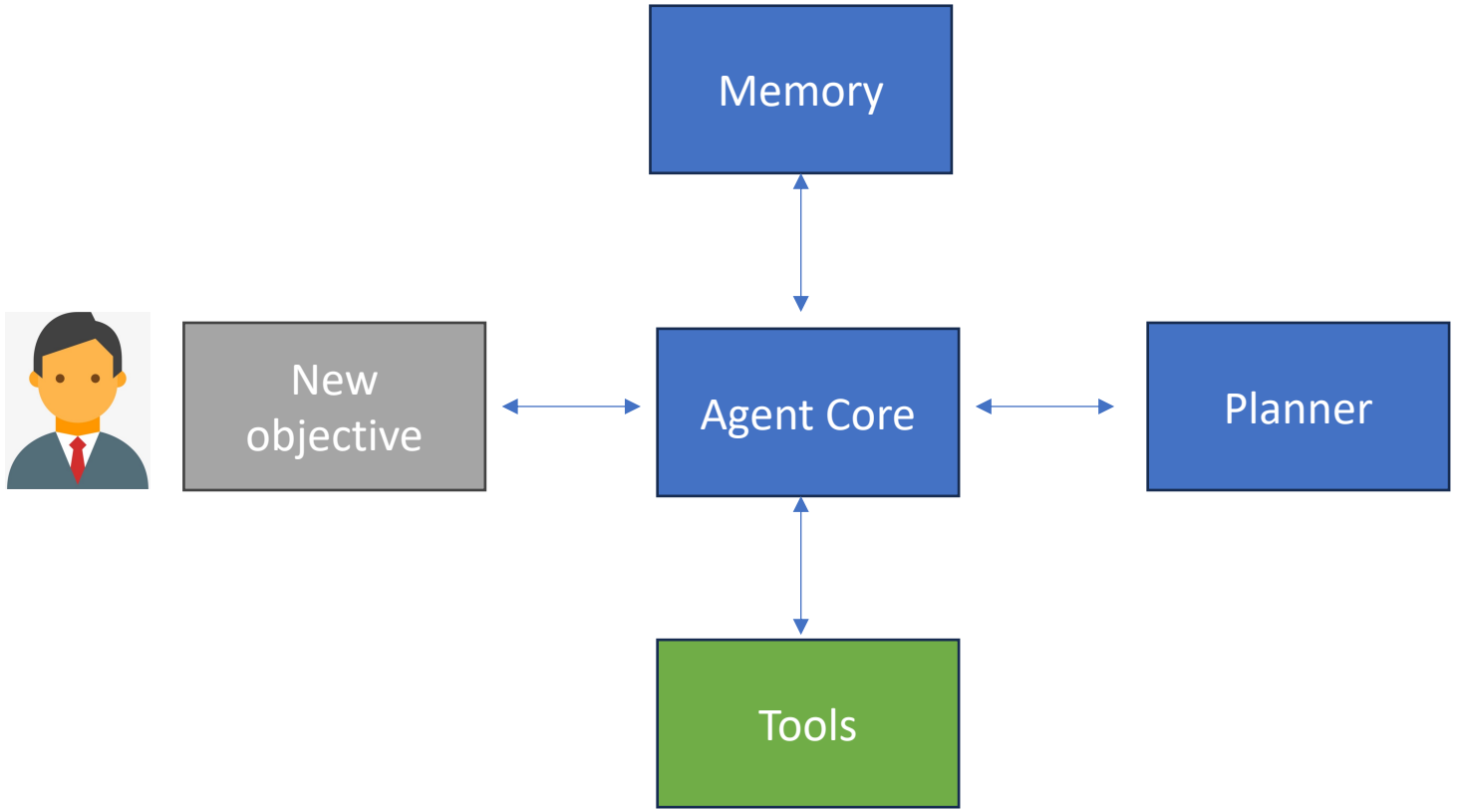
The project

- First part
  - Development of an autonomous agent that will play on your behalf
    - Represents and mange Beliefs from sensing data (including belief revision)
    - Activate goals / Intentions and act on the environment (including intentions revision)
    - Use predefined plans to achieve goals / intentions (parcels are known since the beginning)
  - Extend the above software with automated planning
    - Once an intention is activated the agent must call the planner to have the plan to execute
  - Validation and testing with predefined simulation runs (all together)
- Second part
  - Extend your software including a second LLM-based software agent
    - The LLM-based software agent uses an LLM to interpret the objective, reason about the context, and generate and iteratively refine a plan to achieve the desired outcome.
    - The two agents *can* communicate one another, exchanges beliefs (e.g., beliefs about the environment that other cannot see or beliefs about the intentions one agent is committed to, and so on) and coordinate (e.g., the closest agent will commit to pickup a new parcel)
  - Validation and testing with predefined simulation runs (all together)

© P. Giorgini and M. Robol and M. Bombieri

Second part: what is an LLM-based agent?

We will see it in the next lessons but we can describe it as an agentic system that uses an LLM to **reason through a problem**, **create a plan** to solve the problem, and **execute the plan** with the help of a set of **tools**.



General components of an LLM-based agent.

© P. Giorgini and M. Robol and M. Bombieri

Second part: more details

In the second phase of the game, objectives expressed in natural language will also be introduced.
You will therefore need to implement an LLM-based agent capable of:

1. **Update the context**

- reading and interpreting these objectives expressed in natural language
- observing the environment (and exchanging beliefs with the BDI agent)
- consulting a catalog of available tools that the agent can use to achieve the objective

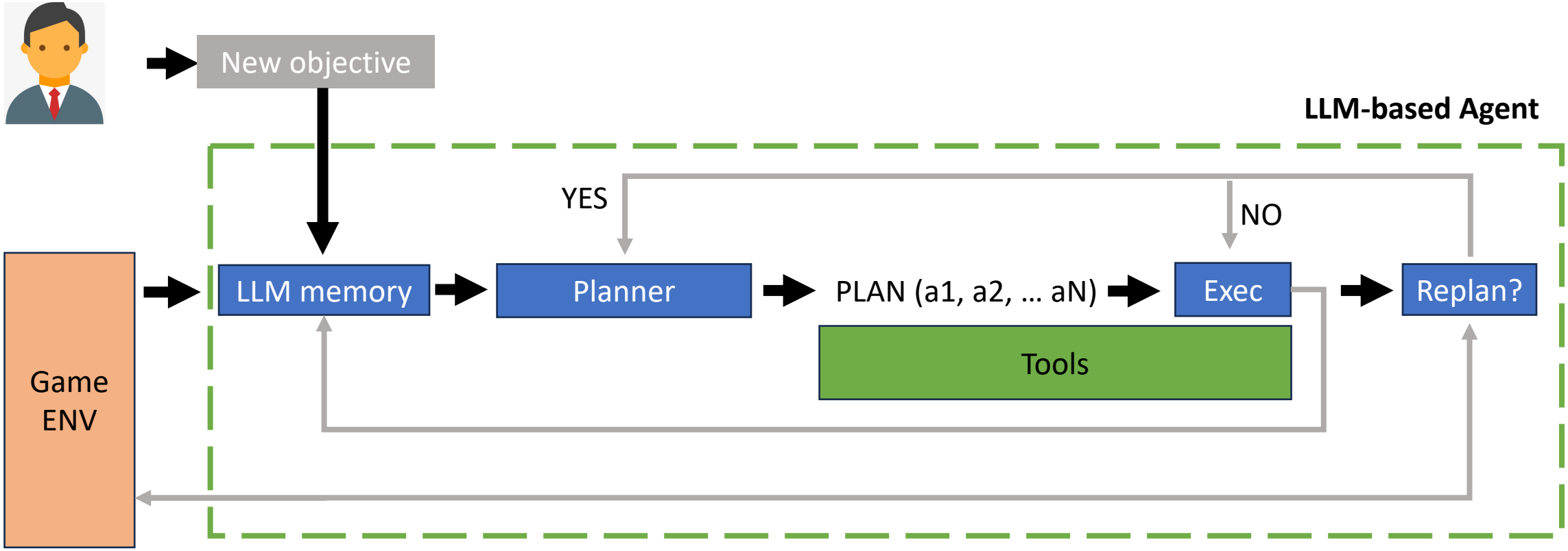
This information forms the LLM memory, which must be dynamically updated, as both the environment and the objectives may change over time.

2. **Define or refine the plan, then act**

- The LLM must then reason over the memory to define an action plan: which sequence of tools should be used to achieve the objective?
- The agent must also be able to execute the plan and adapt it dynamically if observations from the environment change or if the current objective is modified or replaced.

© P. Giorgini and M. Robol and M. Bombieri

Second part: a possible architecture



© P. Giorgini and M. Robol and M. Bombieri

Second part: general components

The one shown in the previous slide is just an example of an architecture. You will be free to implement the LLM-based agent using the architecture you prefer, but the key components that must be there are:

- **LLM-memory**: it is the LLM-context; it contains the objective and the observations from the game environment
- **LLM-Planner**: it decomposes the objective expressed in natural language into a set of actions that can be executed using some tools. It can use techniques such as Chain of Though or reflexion (or others) to improve the reasoning capabilities.
- **LLM-Replanner**: techniques such as ReAct, Reflexion, and Chain-of-Thought reasoning (or others) can be leveraged to iteratively refine and improve the plan generated by the agent, especially if observations from the environment of the goal change.

The list of tools you can use to solve the task is predefined and available on a server.

More details on the techniques and architectures of LLM-based agents will be provided during the course.

© P. Giorgini and M. Robol and M. Bombieri

Second part: the infrastructure

The core game infrastructure will remain the same as in the first part, with the following additions:

- LLMs available on a server, accessible via API using an access token, which you can call from your client
- An extended catalog of tools available on a sever that the LLM agent can query via an API.

© P. Giorgini and M. Robol and M. Bombieri

Project delivery

- Javascript code
- Final Report
  - Max 10 pages, which explains what you have done and how
- Both code and final report will be evaluated for the exam
  - Oral presentation of the project
- We will provide you the link where you can submit your project
  - At least 1 week before the exam (the exact deadlines will be provided to you later in the course)

© P. Giorgini and M. Robol and M. Bombieri

Let’s play (on the cloud)

- Open https://deliveroojs.azurewebsites.net/
  - Put your name and retrieve your freshly generated token
  - Play one against others, let’s see how it works
  - This is a free tier on azure, and it is not really performative when multiple users are connected

© P. Giorgini and M. Robol and M. Bombieri

Let’s play on unitn internal deploy

- You will need to install unitn VPN
  - Follow the instructions on ServiceDesk -> Networking -> VPN-> GlobalProtect https://servicedesk.unitn.it/sd/en/service/vpn?id=unitrento_v2_service_card&sys_id=e5a898fec35bfd104cbb7055df013171
  - You will be required to download GlobalProtect setup **vpn-mfa.icts.unitn.it** provider and login with unitn
- Open
  - https://deliveroojs.bears.disi.unitn.it/
  - Fallback server: https://deliveroojs.rtibdi.disi.unitn.it/

© P. Giorgini and M. Robol and M. Bombieri

Running the game locally

- Download the game from Github
  - Git clone https://github.com/unitn-ASA/Deliveroo.js.git
- Install packages and run the server
  - npm install; npm run build; npm start
- Play from the browser
  - http://localhost:8080 or the ip address of your colleague
- Create an admin token to connect in '**god**' mode, default password when you run the game locally is 'admin'
  - Different maps can be loaded with different configuration, such as, parcel spawning logic and actions duration.

© P. Giorgini and M. Robol and M. Bombieri

Getting started with scripting your NPC

- Have your local Deliveroo.js server running or play on the cloud
- Download the “draft” of scripted agent from Github
  - Git clone https://github.com/unitn-ASA/DeliverooAgent.js
- Install, setup connection parameters, and run the client agent
  - npm install
  - Edit ‘host’ and ‘token’ in config.js (get your token from the 3d client!)
  - `node demo_agent_client.js`
- Check in the browser what the agent is doing

© P. Giorgini and M. Robol and M. Bombieri